

Unification and Matching in Free Abelian Groups

John D. Ramsdell

February 18, 2014

The CPSA authentication test solving algorithm depends on origination assumptions to constrain the adversary. A non-origination assumption asserts that a value is never carried in a message, and so the adversary cannot obtain the value. A unique origination assumption asserts a value is first carried in a transmission exactly once, so if a regular strand originates a value, the adversary cannot. The concept of unique and non-origination work well when the message algebra contains only simple equations, such as the fact that the inverse of the inverse of an asymmetric key is equivalent to the key. There is a simple procedure to determine if the adversary can construct a term given a set of origination assumptions.

In CPSA's implementation of a Diffie-Hellman algebra, exponentiated values have been added. The equational identities added to our algebra are described in [5, Section 4], however for this note, we focus solely on the terms in the exponent. The equational identities AG for Abelian groups axiomatize the exponent. There are efficient algorithms for unification and matching modulo AG [1, Section 5.1]. CPSA uses an algorithm that reduces the problem to finding integer solutions to an inhomogeneous linear equation with integer coefficients. The equation solver used is from The Art of Computer Programming [3, Pg. 327].

One must be very careful about origination assumptions when implementing a Diffie-Hellman algebra, especially in light of the fact that unification and matching are essential ingredients in the authentication test solving algorithm. If a variable has an origination assumption, and terms are normalized, it is easy to check if the adversary can create a term by simply determining if the variable occurs in the term. However, this is not the case for a

compound term with an origination assumption. Unconstrained unification creates the possibility that an origination assumption on a variable turns into an origination assumption on a compound term.

In this note, we propose using sorts to constrain unification and matching so as to force origination assumptions to be on variables only. The idea is to restrict origination assumptions to basis elements of the free algebra. With this restriction, determining what the adversary can construct remains easily computable.

Consider a free Abelian group G with basis B using an order sorted algebra. Our notation is from the chapter on unification in the Handbook of Automated Reasoning [1], and additive notation is used for terms of the group. The binary operation is $+$, constant 0 is the identity element, and $-$ is the unary inverse operation. The binary operation $+$ is left associative so that $x + y + z = (x + y) + z$. The equational identities for an Abelian group are

$x + 0 \approx x$	Unit element	$\mathcal{U}$
$x + -x \approx 0$	Inverse element	$\mathcal{I}$
$x + y \approx y + x$	Commutativity	$\mathcal{C}$
$x + y + z \approx x + (y + z)$	Associativity	$\mathcal{A}$

The normal form of term t is $c_1x_1 + c_2x_2 + \dots + c_nx_n$, where c_i is a non-zero integer, and for $c > 0$, cx abbreviates $x + x + \dots + x$ summed c times while for $c < 0$, cx abbreviates $-((-c)x)$. Every term has a normal form.

In the order sorted signature for our free Abelian group, there are two sorts, G and B with $B < G$, and no operations or equations specific to sort B .

The unsorted algebra isomorphic to the sorted algebra adds a unary inclusion operation b to the signature. For term t , we write $\bar{t}$ for $b(t)$. A term t is *constrained* if it occurs as $\bar{t}$. Because problems in this algebra are derived from the order-sorted algebra, terms have a restricted form. Whenever $\bar{t}$ occurs in a term, t is a variable. Furthermore, when variable x is constrained, it occurs everywhere in the term in the form $\bar{x}$. For example, the term $x + \bar{x}$ is ill-formed. Thus the normal form of a term is $c_1x_1 + c_2x_2 + \dots + c_mx_m + c_{m+1}\bar{x}_{m+1} + c_{m+2}\bar{x}_{m+2} + \dots + c_n\bar{x}_n$.

Combination of Unification Algorithms. Section 6.1 of the Handbook [1] describes a general method for combining unification algorithms. Let A_+ be the theory associated with the Abelian group, and $B_b := \{\bar{x} \approx \bar{x}\}$, so that $E = A_+ \cup B_b$. Obviously, $=_{B_b}$ is just syntactic equality. The “dummy”

identity $\bar{x} \approx \bar{x}$ ensures that b belongs to the signature of B_b .

An E -unification problem is written $t =_E^? t'$. A *reduced* unification problem is one in which t contains no constrained variables, t' contains no unconstrained variables, and t and t' are in normal form. Any E -unification problem can be converted into a reduced E -unification problem.

To convert the reduced problem into decomposed form, a fresh variable v_i is generated for each constrained variable $\bar{x}_i$ in t' . When $t' = c_1\bar{x}_1 + c_2\bar{x}_2 + \dots + c_n\bar{x}_n$, the decomposed form is

$$\{t - c_1v_1 - c_2v_2 - \dots - c_nv_n \stackrel{?}{=}_{A_+} 0, v_1 \stackrel{?}{=}_{B_b} \bar{x}_1, v_2 \stackrel{?}{=}_{B_b} \bar{x}_2, \dots, v_n \stackrel{?}{=}_{B_b} \bar{x}_n\},$$

where $x - cy$ abbreviates $x + (-c)y$.

For this problem, the shared variables are the fresh variables v_i introduced during decomposition. Each shared variable is labeled with theory B_b . The ordering of shared variables is irrelevant because of the simple nature of the equations in theory B_b . The only source of non-determinism is the partitioning of the shared variables.

Example 1.

$$\begin{array}{ll} \{z \stackrel{?}{=} \bar{x} - \bar{y}\} & \\ \{z - v_0 + v_1 \stackrel{?}{=}_{A_+} 0, v_0 \stackrel{?}{=}_{B_b} \bar{x}, v_1 \stackrel{?}{=}_{B_b} \bar{y}\} & \text{Decomposed form} \\ \{\{v_0\}, \{v_1\}\} & \text{Partition} \\ \{z \approx \bar{x} - \bar{y}, v_0 \approx \bar{x}, v_1 \approx \bar{y}\} & \text{Solution} \end{array}$$

Example 2.

$$\begin{array}{ll} \{0 \stackrel{?}{=} \bar{w} + \bar{x} - \bar{y} - \bar{z}\} & \\ \{v_0 + v_1 - v_2 - v_3 \stackrel{?}{=}_{A_+} 0, v_0 \stackrel{?}{=}_{B_b} \bar{w}, v_1 \stackrel{?}{=}_{B_b} \bar{x}, v_2 \stackrel{?}{=}_{B_b} \bar{y}, v_3 \stackrel{?}{=}_{B_b} \bar{z}\} & \text{Decomposed form} \\ \{\{v_0, v_2\}, \{v_1, v_3\}\} & \text{Partition} \\ \{w \approx y, x \approx z, \dots\} & \text{Solution} \end{array}$$

To find the other solution, $\{w \approx z, x \approx y\}$, use the partition $\{\{v_0, v_3\}, \{v_1, v_2\}\}$.

Efficient Unification Algorithm. The issue surrounding an efficient implementation is the fast generation of the fewest variable partitions required to generate a complete set of unifiers. A means to reduce the amount of

non-determinism in this algorithm is reported in [2]. The paper provides a method to intertwine equation solving with making decisions about non-deterministic choices.

The only kinds of decisions used here are equality/disequality decisions for constrained variables. For constrained variables x and y , we write $x \doteq y$ for the decision to equate x and y , and $x \neq y$ for the decision to keep them distinct.

Let σ and θ be substitutions, and D be a set of decisions. To compute a complete set of unifiers for problem $t =^? t'$, compute $unify(t =^? t', \emptyset, Id)$ where $unify(t =^? t', D, \theta)$ is

1. if σ is an A_+ -unifier for $t =^? t'$ obtained by treating constrained variables as constants then return $\{\sigma\theta\}$;
2. else return $unify'(t =^? t', D, \theta)$.

Function $unify'(t =^? t', D, \theta)$ is

1. if all distinct pairs of constrained variables have a decision in D then return θ ;
2. else choose variables x and y without a decision in D and return

$$unify(t =^? t' \{x \mapsto y\}, D \cup \{x \doteq y\}, \theta \{x \mapsto y\}) \cup \\ unify'(t =^? t', D \cup \{x \neq y\}, \theta).$$

The algorithm can further be improved by changing function $unify$ so that when A_+ -unification fails, the results of the steps up to the failure are applied to the recursive call so as to avoid repeating these steps. This refinement is due to [4, Chapter 8]. To justify this refinement, one must show that constrained variable identification does not invalidate any previous A_+ -unification steps.

To implement this refinement, we assume the equation is in reduced form and replace $unify$ with $unify_0$, which expands the A_+ -unification algorithm. For equation, $t =^? t'$ let $t = \sum_{i=1}^n c_i x_i$ and $t' = \sum_{j=1}^m d_j \bar{y}_j$.

Function $unify_0(t =^? t', D, \theta)$ is

1. let c_i be the smallest coefficient in absolute value;
2. if $c_i < 0$ then return $unify_0(-t =^? -t', D, \theta)$;

3. else if $c_i = 1$ then return $\{\sigma\theta\}$ where $\sigma = \{x_i \mapsto \sum_j^m d_j \bar{y}_j - \sum_{k \neq i}^n c_k x_k\}$;
4. else if c_i divides every coefficient in c then
 - (a) if c_1 divides every coefficient in d then divide c and d by c_i and goto step 3;
 - (b) else return $unify'(t = ? t', D, \theta)$;
5. else eliminate x_i in favor of freshly created variable x_{n+1} using $\sigma = \{x_i \mapsto x_{n+1} - \sum_{k \neq i} (c_k/c_i)x_k\}$ and then return $unify_0(t'' = ? t', D, \sigma\theta)$ where $t'' = c_i x_{n+1} + \sum_k (c_k \bmod c_i)x_k$.

Constrained variable identification does not invalidate any previous A_+ -unification steps because $\gcd(d_i, d_j)$ divides $d_i + d_j$.

There is another opportunity for performance improvement by refining function $unify'$. It could inspect the coefficients of constrained variables when deciding which constrained variables to identify. This refinement has yet to be explored.

References

- [1] Franz Baader and Wayne Snyder. Unification theory. In Alan Robinson and Andrei Voronkov, editors, *Handbook of Automated Reasoning*, volume 1, chapter 8. The MIT Press, 2001. <http://www.cs.bu.edu/~snyder/publications/UnifChapter.pdf>.
- [2] Stephan Kepser and Jörn Richts. Optimisation techniques for combining constraint solvers. In Maarten de Rijke and Dov Gabbay, editors, *Frontiers of Combining Systems 2*, pages 193–210, Asmsterdam, 1999. Research Studies Press/Wiley. <http://tcl.sfs.uni-tuebingen.de/~kepsers/papers/optimisation.ps.gz>.
- [3] Donald E. Knuth. *The Art of Computer Programming, Volume II: Seminumerical Algorithms, 2nd Edition*. Addison-Wesley, 1981.
- [4] Zhiqiang Liu. *Dealing Efficiently with Exclusive OR, Abelian Groups and Homomorphism in Cryptographic Protocol Analysis*. PhD thesis, Clarkson University, September 2012. <http://people.clarkson.edu/~clynch/papers/>.

- [5] Catherine Meadows and Paliath Narendran. A unification algorithm for the group diffie-hellman protocol. In *WITS 2002*, January 2002. http://www.dsi.unive.it/IFIPWG1_7/WITS2002/prog/annotated_program.html.